UCC 103 – PRINCIPLES OF COMPUTING

Chapter 4: Data Representation and Computer Files Systems

4.1. Introduction

Data Representation refers to the methods used internally to represent information stored in a computer. Computers store lots of different types of information:

- Numbers:
- Text
- Graphics of many varieties (stills, video, animation)
- Sound

At least, these all seem different to us. However, **ALL** types of information stored in a computer are stored internally in the same simple format: a sequence of 0's and 1's

Computers work with a binary number system that consists of only two digits - zero and one. Inside the computer binary number is represented by an electrical pulse. One means a pulse of electricity and zero means no pulse. All the data entered into computers is first converted into the binary number system. One digit in binary number system is called bit and combination of eight bits is called byte. A byte is the basic unit that is used to represent the alphabetic, numeric and alphanumeric data.

4.2. Types of Data

Data is the combination of characters, numbers and symbols collected for a specific purpose. Data is divided into three types;

1) Alphabetic data is used to represent 26 alphabetic. It consist of capital letters from A to Z, small letters from a to z and blank space. Alphabetic data is also called non numerical data.
2) Numeric data consist of ten digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, two signs + and - and decimal point. There are different types of number system that are used to represent numeric data. These number systems are decimal number system, binary number system, octal number system and hexadecimal number system.
3) Alphanumeric data. Combines both Numeric and Alphabetic numbers as well as special symbols.

4.3. Data Representation

1) **Numbers.** Assigned a numeric number
2) **Text:** Text can be represented easily by assigning a unique numeric value for each symbol used in the text. For example, the widely used ASCII code (American Standard Code for Information Interchange) defines 128 different symbols (all the characters found on a standard keyboard, plus a few extra), and assigns to each a unique numeric code between 0 and 127. In ASCII, an "A" is 65," B" is 66, "a" is 97, "b" is 98, and so forth. When you save a file as "plain text", it is stored using ASCII. ASCII format uses 1 byte per character 1 byte gives only 256 (128 standard and 128 non-standard) possible characters The code value for any character can be converted to base 2, so any written message made up of ASCII characters can be converted to a string of 0's and 1's.

3) **Graphics:** Graphics that are displayed on a computer screen consist of pixels: the tiny "dots" of color that collectively "paint" a graphic image on a computer screen. The pixels are organized into many rows on the screen. In one common configuration, each row is 640 pixels long, and there are 480 such rows. Another configuration (and the one used on the screens in the lab) is 800 pixels per row with 600 rows, which is referred to as a "resolution of 800x600." Each pixel has two properties: its location on the screen and its color.

4.4. Data Types

A **data type** or simply **type** is a classification identifying one of various types of data, that determines the possible values for that type; the operations that can be done on values of that type; the meaning of the data; and the way values of that type can be stored

Primitive data types

A **primitive data type** is either of the following

- a *basic type* is a data type provided by a Programming language as a basic building block. Most languages allow more complicated composite types to be recursively constructed starting from basic types.
- a *built-in type* is a data type for which the programming language provides built-in support.

In most programming languages, all basic data types are built-in. In addition, many languages also provide a set of composite data types. Opinions vary as to whether a built-in type that is not basic should be considered "primitive". The actual range of primitive data types that is available is dependent upon the specific programming language that is being used.

Classic basic primitive types may include:

1. **Character (character, char);**
   A character type (typically called "char") may contain a single letter, digit, punctuation marks, symbol, formatting code, control code, or some other specialized code.
2. **Integer** (integer, int, short, long, byte) with a variety of Precisions;
   an **integer** is a datum of **integral data type**, a data type which represents some finite subset of the mathematical integers. Integral data types may be of different sizes and may or may not be allowed to contain negative values. Integers are commonly represented in a computer as a group of binary digits
   An integer data type can hold a whole number, but no fraction. Integers may be either signed (allowing negative values) or unsigned (nonnegative values only
   - Literals for integers consist of a sequence of digits
   - Negation is indicated by a minus sign (−) before the value
     - 42
     - 10000
     - −233000
3. **Floating Point number** (float, double, real, double precision);
   A floating-point number represents a limited-precision rational number that may have a fractional part. example

- 20.0005
- 99.9

4. **Boolean,** logical values **true** and **false**.
   A Boolean type, typically denoted "bool" or "boolean", is typically a *logical type* that can be either "true" or "false". Although only one bit is necessary to accommodate the value set "true" and "false", programming languages typically implement boolean types as one or more bytes.

4.5. How information is stored in computers

Data is represented inside a computer as a series of on and off pulses. Humans think of those pulses in terms of a binary-based numbering system.

Information is stored in computers in the form of **bits**. A bit is used to represent information in the computer. They are referred to as binary digits i.e. the 0's and 1's with 0 representing an OFF state and 1 representing an ON state. The stored bits are usually retrieved from computers memory for manipulation by the processor

A single bit alone cannot represent a number, letters or special characters, to represent information; bits are combined into groups of eight. A group of eight bits is called a **byte**. Each byte can be used to represent a number, letter or special character.

Binary Numbers

Normally we write numbers using digits 0 to 9. This is called base 10. However, any positive integer (whole number) can be easily represented by a sequence of 0's and 1's. Numbers in this form are said to be in base 2 and they are called binary numbers. Base 10 numbers use a positional system based on powers of 10 to indicate their value. The number 123 is really 1 hundred + 2 tens + 3 ones. The value of each position is determined by ever-higher powers of 10, read from left to right. Base 2 works the same way, just with different powers. The number 101 in base 2 is really 1 four + 0 twos + 1 one (which equals 5 in base 10).

4.6. Computer Files System

A **computer file** is a resource for storing information, which is available to a computer program and is usually based on some kind of durable storage.

4.6.1. File Contents

A computer file must have a **file name**. On most modern operating systems, files are organized into one-dimensional arrays of bytes. The format of a file is defined by its content since a file is solely a container for data, although, on some platforms the format is usually indicated by its filename extension, specifying the rules for how the bytes must be organized and interpreted meaningfully.

For example,

- .txt – plain text files -
- .doc/ .docx – word processing file
- .xls – spreadsheet file (excel)

- .pdf – portable document format
- .exe – executable file

NB. A computer file must have a **file name** and an **extension** that indicates the content of the file.

4.6.2. File Size

File size measures the size of a computer file. Typically it is measured in bytes and indicates how much storage is associated with the file. The actual amount of disk space consumed by the file depends on the file system. The maximum file size a file system supports depends on the number of bits reserved to store size information and the total size of the file system. Some common file size units are:

- 1 byte = 8 bit
- 1 **KiB** = 1,024 bytes
- 1 **MiB** = 1,048,576 bytes
- 1 **GiB** = 1,073,741,824 bytes
- 1 **TiB** = 1,099,511,627,776 bytes

4.6.3. Organizing the data in a file

Information in a computer file can consist of smaller packets of information (often called "records" or "lines") that are individually different but share some common traits. For example, a payroll file might contain information concerning all the employees in a company and their payroll details; each record in the payroll file concerns just one employee, and all the records have the common trait of being related to payroll—this is very similar to placing all payroll information into a specific filing cabinet in an office that does not have a computer. A text file may contain lines of text, corresponding to printed lines on a piece of paper.

The way information is grouped into a file is entirely up to how it is designed. This has led to a plethora of more or less standardized file structures for all imaginable purposes, from the simplest to the most complex. Most computer files are used by computer programs which create, modify or delete the files for their own use on an as-needed basis. The programmers who create the programs decide what files are needed, how they are to be used and (often) their names. In some cases, computer programs manipulate files that are made visible to the computer user. For example, in a word-processing, the user manipulates document files that the user personally names. Although the content of the document file is arranged in a format that the word-processing program understands, the user is able to choose the name and location of the file and provide the bulk of the information (such as words and text) that will be stored in the file.

Many applications pack all their data files into a single file called archive file, using internal markers to discern the different types of information contained within. The benefits of the archive file are to lower the number of files for easier transfer, to reduce storage usage, or just to organize outdated files. The archive file must often be unpacked before next using.

4.7. File Operations

The most basic operations that programs can perform on a file are:

- Create a new file
- Change the access permissions and attributes/characteristics of a file
  - ✓ Access permissions – rights on how the users can use the file
  - ✓ File attributes are metadata associated with computer files that define file system behavior. Each attribute can have one of two states: set and cleared
- Open a file, which makes the file contents available to the program
- Read data from a file
- Write data to a file
- Close a file, terminating the association between it and the program

4.8. Computer File Systems

A *filesystem* is the methods and data structures that an operating system uses to keep track of files on a disk or partition; that is, the way the files are organized on the disk. It is a method for storing and organizing computer files and the data they contain to make it easy to find and access them. A file system is used to control how information is stored and retrieved. A file system is a set of abstract data types that are implemented for the storage, hierarchical organization, manipulation, navigation, access, and retrieval of data.

Most computers have at least one file system. Some computers allow the use of several different file systems.

File systems are used to implement type of data store to store, retrieve and update a set of file. Without a file system, information placed in a storage area would be one large body of information with no way to tell where one piece of information stops and the next begins. File systems may use a data storage device such as a hard disk or CD-ROM and involve maintaining the physical location of the files, or they may be virtual and exist only as an access method for virtual data or for data over a network (e.g. NFS).

The file system manages access to both the content of files and the metadata about those files. It is responsible for arranging storage space; reliability, efficiency, and tuning with regard to the physical storage medium are important design considerations.

4.9. Functions of File System

a). Space Management

File systems allocate space in a granular manner, usually multiple physical units on the device. The file system is responsible for organizing files and directories, and keeping track of which areas of the media belong to which file and which are not being used.

File System Fragmentation

File system fragmentation occurs when unused space or single files are not contiguous. As a file system is used, files are created, modified and deleted. When a file is created the file system allocates space for the data. Some file systems permit or require specifying an initial

space allocation and subsequent incremental allocations as the file grows. As files are deleted the space they were allocated eventually is considered available for use by other files. This creates alternating used and unused areas of various sizes. This is free space fragmentation. When a file is created and there is not an area of contiguous space available for its initial allocation the space must be assigned in fragments. When a file is modified such that it becomes larger it may exceed the space initially allocated to it, another allocation must be assigned elsewhere and the file becomes fragmented.

b). Restricting and permitting access

There are several mechanisms used by file systems to control access to data. Usually the intent is to prevent reading or modifying files by a user or group of users. Another reason is to ensure data is modified in a controlled way so access may be restricted to a specific program. Examples include passwords stored in the metadata of the file or elsewhere and file permissions in the form of permission bits, access control lists, or capabilities. The need for file system utilities to be able to access the data at the media level to reorganize the structures and provide efficient backup usually means that these are only effective for polite users but are not effective against intruders.

Methods for encrypting file data are sometimes included in the file system. This is very effective since there is no need for file system utilities to know the encryption seed to effectively manage the data. The risks of relying on encryption include the fact that an attacker can copy the data and use brute force to decrypt the data. Losing the seed means losing the data.

c). Maintaining integrity

One significant responsibility of a file system is to ensure that, regardless of the actions by programs accessing the data, the structure remains consistent. This includes actions taken if a program modifying data terminates abnormally or neglects to inform the file system that it has completed its activities. This may include updating the metadata, the directory entry and handling any data that was buffered but not yet updated on the physical storage media. Other failures which the file system must deal with include media failures or loss of connection to remote systems.

In the event of an operating system failure or "soft" power failure, special routines in the file system must be invoked similar to when an individual program fails. The file system must also be able to correct damaged structures. These may occur as a result of an operating system failure for which the OS was unable to notify the file system, power failure or reset. The file system must also record events to allow analysis of systemic issues as well as problems with specific files or directories.

d). Manage User data

The most important purpose of a file system is to manage user data. This includes storing, retrieving and updating data. Some file systems accept data for storage as a stream of bytes which are collected and stored in a manner efficient for the media. When a program retrieves

the data it specifies the size of a memory buffer and the file system transfers data from the media to the buffer. Sometimes a runtime library routine may allow the user program to define a *record* based on a library call specifying a length. When the user program reads the data the library retrieves data via the file system and returns a *record*.

Some file systems allow the specification of a fixed record length which is used for all write and reads. This facilitates updating records.

An identification for each record, also known as a key, makes for a more sophisticated file system. The user program can read, write and update records without regard with their location. This requires complicated management of blocks of media usually separating key blocks and data blocks. Very efficient algorithms can be developed with pyramid structure for locating records.

4.10. Types of File Systems

File system types can be classified into disk/tape file systems, network file systems and special-purpose file systems.

Disk file systems

Disk file systems are file systems which manage data on permanent storage devices, A *disk file system* takes advantages of the ability of disk storage media to randomly address data in a short amount of time. Additional considerations include the speed of accessing data following that initially requested and the anticipation that the following data may also be requested. This permits multiple users (or processes) access to various data on the disk without regard to the sequential location of the data.
Examples; File Allocation Table (FAT) New Technology File System (NTFS)

Flash file systems

A *flash file system* considers the special abilities, performance and restrictions of flash memory devices. Frequently a disk file system can use a flash memory device as the underlying storage media but it is much better to use a file system specifically designed for a flash device.

Tape file systems

A *tape file system* is a file system and tape format designed to store files on tape in a self-describing form. Magnetic tapes are sequential storage media with significantly longer random data access times than disks, posing challenges to the creation and efficient management of a general-purpose file system.

Database file systems

Another concept for file management is the idea of a database-based file system. Instead of, or in addition to, hierarchical structured management, files are identified by their characteristics, like type of file, topic, author, or similar rich metadata

Transactional file systems

Some programs need to update multiple files "all at once". For example, a software installation may write program binaries, libraries, and configuration files. If the software installation fails, the program may be unusable. Transaction file systems creates temporary files that keeps records of the current transactions. The transaction files are used to update the master files.

Transaction processing introduces the isolation guarantee, which states that operations within a transaction are hidden from other threads on the system until the transaction commits, and that interfering operations on the system will be properly serialized with the transaction. Transactions also provide the atomicity guarantee, that operations inside of a transaction are either all committed, or the transaction can be aborted and the system discards all of its partial results. This means that if there is a crash or power failure, after recovery, the stored state will be consistent. Either the software will be completely installed or the failed installation will be completely rolled back, but an unusable partial install will not be left on the system.

Ensuring consistency across multiple file system operations is difficult, if not impossible, without file system transactions. File locking can be used as a concurrency control mechanism for individual files, but it typically does not protect the directory structure or file metadata. For instance, file locking cannot prevent race conditions on symbolic links. File locking also cannot automatically roll back a failed operation, such as a software upgrade; this requires atomicity.

Journaling file systems are one technique used to introduce transaction-level consistency to file system structures. Journal transactions are not exposed to programs as part of the OS API; they are only used internally to ensure consistency at the granularity of a single system call.

Data backup systems typically do not provide support for direct backup of data stored in a transactional manner, which makes recovery of reliable and consistent data sets difficult. Most backup software simply notes what files have changed since a certain time, regardless of the transactional state shared across multiple files in the overall dataset. As a workaround, some database systems simply produce an archived state file containing all data up to that point, and the backup software only backs that up and does not interact directly with the active transactional databases at all. Recovery requires separate recreation of the database from the state file, after the file has been restored by the backup software.

Network file systems

The Network File System, or NFS, is a distributed file system that allows you to access files and directories located on remote computers and treat those files and directories as if they were local. For example, you can use operating system commands to create, remove, read, write, and set file attributes for remote files and directories. A *network file system* is a file system that acts as a client for a remote file access protocol, providing access to files on a server.

Features

a) Access transparency is that clients are unaware that files are distributed and can access them in the same way as local files are accessed.

b) Location transparency A consistent name space exists encompassing local as well as remote files. The name of a file does not give its location.
c) Concurrency transparency All clients have the same view of the state of the file system. This means that if one process is modifying a file, any other processes on the same system or remote systems that are accessing the files will see the modifications in a coherent manner.
d) Failure transparency The client and client programs should operate correctly after a server failure.
e) Heterogeneity File service should be provided across different hardware and operating system platforms.
f) Scalability The file system should work well in small environments (1 machine, a dozen machines) and also scale gracefully to huge ones (hundreds through tens of thousands of systems).
g) Replication transparency To support scalability, we may wish to replicate files across multiple servers. Clients should be unaware of this.
h) Migration transparency Files should be able to move around without the client's knowledge.